

Topic 42 — RAG: Retrieval Augmented Generation

Give the model an open book — pipeline, chunking, embeddings, and production reality.

 Core Idea

 The Pipeline

 Production Reality

 Interview Q&A

Layer 1 — The core idea

The problem RAG solves

Ask your local model something about your own company:

You: "What's our refund policy for enterprise customers?"
LLM: "I don't have information about your company's specific policies."

Of course not. The model was trained on public internet text, months or years ago. It has never seen:

- Your company's documents
- Your database
- Anything that happened after training
- Anything private, internal, or newly written

Rooted in Topic 40

Three separate limitations, one cause: **the knowledge is baked into the weights** — and the weights are frozen.

The one-line answer

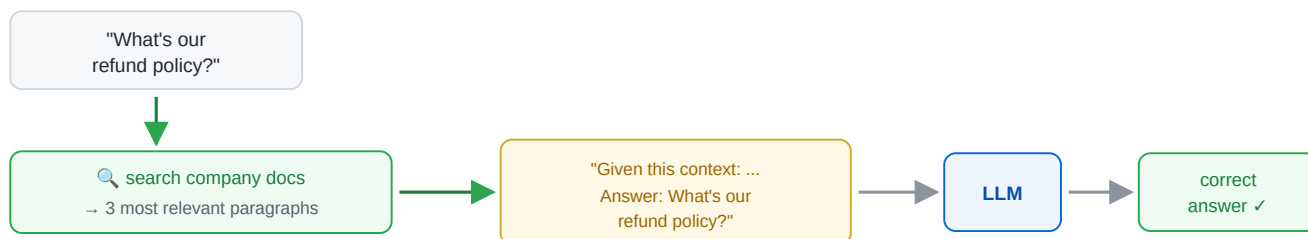
One line

RAG means: look up relevant information first, then paste it into the prompt before asking the question.

WITHOUT RAG



WITH RAG



Same model, same question — the only difference is what's in the prompt.

The model didn't learn anything. You handed it the answer material and asked it to read.

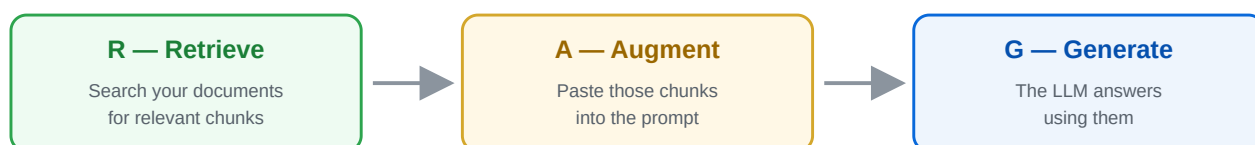
The analogy

Closed-book exam vs open-book exam.

A plain LLM sits a **closed-book** exam — everything from memory, no lookups, confidently wrong when it doesn't know.

RAG gives it the **textbook, already open to the right page**. Same student, same intelligence, dramatically better answers — because the facts are in front of it rather than half-remembered.

The three-step shape



Every RAG system does exactly these three things, no matter how complex.

The simplest possible version

Before any vector databases, understand this — it *is* RAG:

```
# A crude "retriever"
def find_relevant(question: str, documents: list[str]) -> list[str]:
    words = set(question.lower().split())
    scored = [
        (len(words & set(doc.lower().split())), doc)
        for doc in documents
    ]
    scored.sort(reverse=True)
    return [doc for _, doc in scored[:3]]

documents = [
    "Enterprise customers get a 60-day refund window.",
    "Our office is in Pune, Maharashtra.",
    "Standard refunds are processed within 14 days.",
]
```

```

question = "What is the refund policy?"
context = "\n".join(find_relevant(question, documents))

response = client.chat(model="llama3.2", messages=[
    {
        "role": "system",
        "content": f"Answer using ONLY this context:\n\n{context}",
    },
    {"role": "user", "content": question},
])

```

Retrieve → augment → generate. Keyword matching is a *bad* retriever, but **the architecture is complete**. Everything in Layer 2 is about replacing that `find_relevant` function with something that actually works.

Why keyword matching fails

🔥 Ask: "How do I get my money back?"

The document says "refund policy." **Zero shared words**. Keyword search returns nothing.

Humans know those mean the same thing. Keyword matching doesn't. **You need search by meaning, not by words** — which is what embeddings do.

Embeddings, plainly

💡 Definition

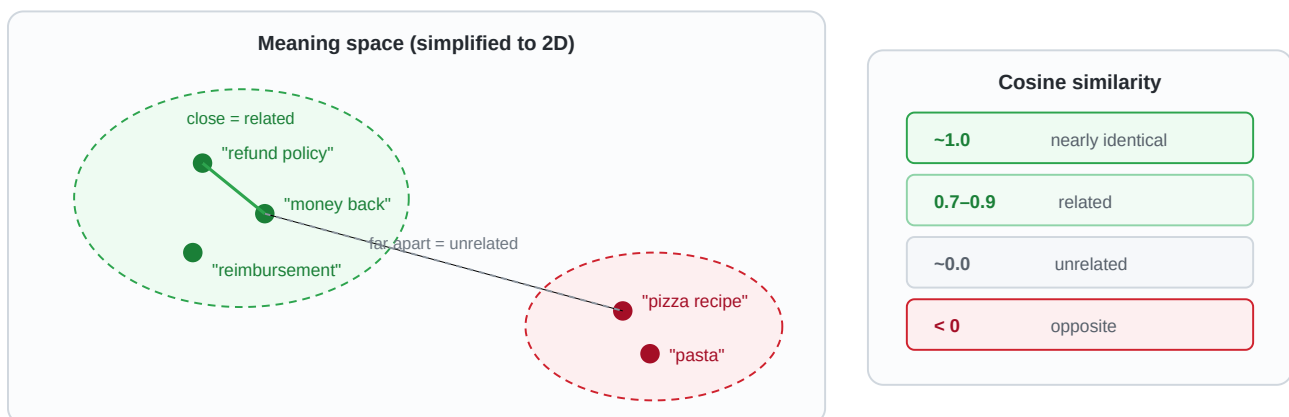
An embedding turns text into a list of numbers that represents its meaning.

```

embed("refund policy")      → [ 0.23, -0.81, 0.44, ... ] # 768 numbers
embed("how to get money back") → [ 0.25, -0.79, 0.41, ... ] # very similar!
embed("pizza recipe")        → [-0.62, 0.11, 0.90, ... ] # very different

```

Similar meaning → similar numbers. The model producing these is an **embedding model**, different from a chat model — it outputs vectors instead of text.



Related meanings cluster together. You don't implement this — a vector database does it for you.

```
ollama pull nomic-embed-text
```

```

result = client.embeddings(model="nomic-embed-text", prompt="refund policy")
print(len(result["embedding"])) # 768

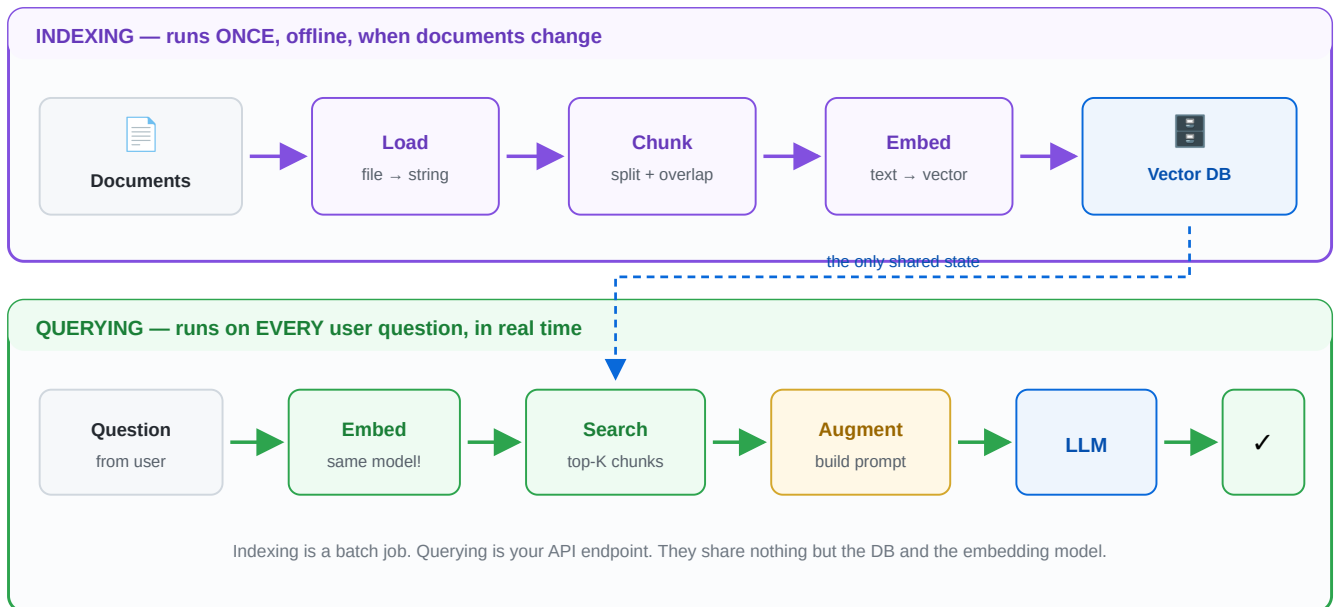
```

That's an embedding model running locally — same Ollama, different model type.

Layer 2 — The pipeline

The two phases

This is the structural thing to internalize: RAG has **two phases that run at completely different times**.



The most common beginner mistake

Confusing the two phases. **Indexing is a batch job you run when documents change. Querying is your API endpoint.**

And the rule that breaks systems silently: **the same embedding model must be used in both phases**. Index with `nomic-embed-text` and query with a different model, and your vectors are in incompatible spaces — garbage results, *no error message*.

Indexing, step by step

Step 1 — Load

```
from pypdf import PdfReader

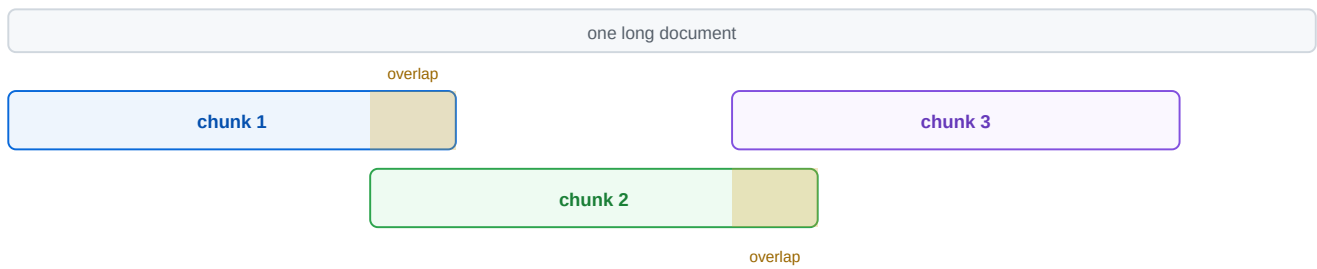
reader = PdfReader("policy.pdf")
text = "\n".join(page.extract_text() for page in reader.pages)
```

Different loaders for PDF, DOCX, HTML, Markdown, database rows. LangChain ships a hundred of them, but they all do one thing: **file in, string out**.

Step 2 — Chunk (the step that decides everything)

You cannot embed a 200-page document as one vector. You must split it.

Chunking with overlap



Without overlap, a sentence split across the boundary becomes unretrievable in both chunks.

```
def chunk_text(text: str, size: int = 500, overlap: int = 50) -> list[str]:
    chunks = []
    start = 0
    while start < len(text):
        chunks.append(text[start:start + size])
        start += size - overlap      # step back to create overlap
    return chunks
```

Chunk size	Trade-off
Too small (~100 chars)	Loses context. "It expires after 30 days" — what does?
Too large (~5000 chars)	Retrieves irrelevant text, wastes context window, dilutes the signal
Sweet spot (~500–1000)	Enough context, still focused

Better than fixed-size splitting is **recursive splitting** — break on paragraphs first, then sentences, then words, only falling back to hard character cuts:

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=100,
    separators=["\n\n", "\n", ". ", " ", ""],
)
chunks = splitter.split_text(text)
```

🚨 Chunking is where most RAG systems fail

If retrieval quality is bad, **fix chunking before you touch anything else**. Bad chunks cannot be rescued by a better model.

Step 3 — Embed

```
embeddings = [
    client.embeddings(model="nomic-embed-text", prompt=chunk)["embedding"]
    for chunk in chunks
]
```

For thousands of chunks, batch this — it's the slow part of indexing.

Step 4 — Store

```
import chromadb

db = chromadb.PersistentClient(path="./chroma_db")
collection = db.get_or_create_collection("company_docs")
```

```
collection.add(
    ids=[f"chunk_{i}" for i in range(len(chunks))],
    documents=chunks,
    embeddings=embeddings,
    metadatas=[{"source": "policy.pdf", "page": 1} for _ in chunks],
)
```

✓ Don't skip metadatas

It's how you **filter** ("only 2026 documents") and how you **cite sources** in the answer. Both matter more than they seem now.

Querying, step by step

```
def answer(question: str) -> str:
    # 1. Embed the question – SAME model as indexing
    q_vec = client.embeddings(
        model="nomic-embed-text", prompt=question
    )["embedding"]

    # 2. Search
    results = collection.query(query_embeddings=[q_vec], n_results=4)
    chunks = results["documents"][0]

    # 3. Build the prompt
    context = "\n\n---\n\n".join(chunks)

    system = f"""Answer the question using ONLY the context below.
    If the context doesn't contain the answer, say "I don't know based on
    the available documents." Do not use outside knowledge.

    CONTEXT:
    {context}"""

    # 4. Generate
    response = client.chat(model="llama3.2", messages=[
        {"role": "system", "content": system},
        {"role": "user", "content": question},
    ])
    return response.message.content
```

⚠ Two lines matter more than they look

"Using ONLY the context" — without it, the model blends retrieved facts with its training data and you can't tell which is which.

"Say I don't know" — without it, the model *invents* an answer when retrieval fails. Giving it explicit permission to decline is one of the highest-value single lines in the whole pipeline. Phase 3 prompt engineering, applied where it counts.

Vector databases

Database	Best for
Chroma	Learning, local dev. <code>pip install</code> , zero config.
FAISS	In-memory, very fast, no persistence layer

Database	Best for
pgvector	You already know Postgres. One database for everything.
Qdrant / Weaviate	Production, self-hosted, good filtering
Pinecone	Managed, no ops

For your stack, **pgvector** is the strong choice — a Postgres extension, so documents, users, and vectors live in one database with real transactions. Coming from Prisma and Mongo, one datastore beats two.

Use cases

Use case	Why RAG fits
Company knowledge base	Internal docs the model never saw
Customer support bot	Answers grounded in your actual policies
Chat with a PDF	Documents uploaded at runtime
Legal / medical research	Citations required, hallucination unacceptable
Codebase Q&A	"How does our auth flow work?"
News / current events	Post-cutoff information
Personalized agents	Retrieve a specific user's history

RAG vs fine-tuning

	RAG	Fine-tuning
Teaches	Facts	Style, format, behaviour
Update cost	Re-index a doc — minutes	Retrain — hours/days + GPUs
Citations	Yes, you know the source	No
Hallucination	Reduced	Not reduced
Access control	Per-document, easy	Impossible — it's in the weights
Cost	Cheap	Expensive

✓ The rule

RAG for knowledge, fine-tuning for behaviour. "Know our product catalog" → RAG. "Always answer in our brand voice as structured JSON" → fine-tuning. They combine well.

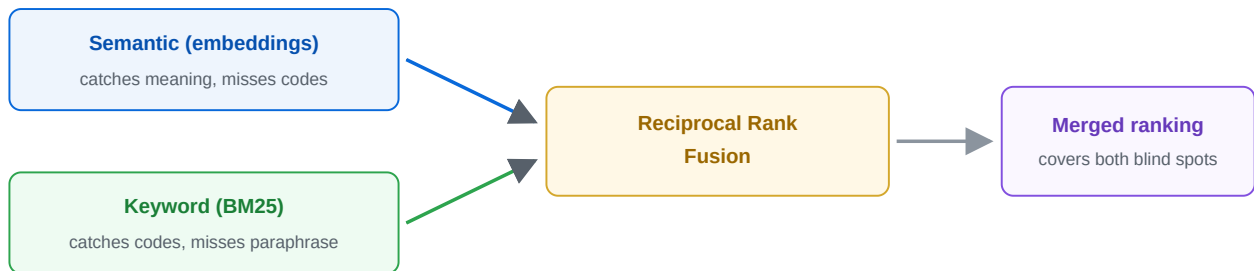
Layer 3 — Making it actually work

Naive RAG breaks in predictable ways

Failure	Cause	Fix
Retrieves irrelevant chunks	Bad chunking or weak embedding model	Fix chunking first
Misses obvious keywords	Semantic search misses exact terms, IDs, product codes	Hybrid search
Right chunk, wrong answer	Buried among distractors	Reranking
Vague question, poor retrieval	Query doesn't match document phrasing	Query rewriting
Needs 2 documents combined	Single retrieval can't reason across docs	Multi-hop / agentic RAG
Confident wrong answer	No grounding instruction	Prompt + citations

Hybrid search

Semantic search fails on exact terms. Search for `"error TX-4471"` and embeddings return "chunks about errors" — not that specific code.

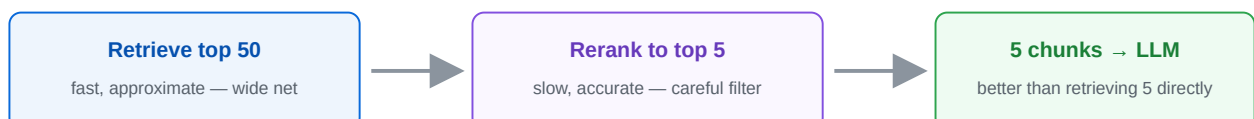


✓ The highest-value upgrade over naive RAG

Most production systems use hybrid search. If you only add one thing beyond the basic pipeline, add this.

Reranking

Retrieval is optimised for speed across millions of chunks, so it's approximate. A **cross-encoder** reranker reads the question and each chunk *together* and scores relevance properly:



Query rewriting

User questions are messy. `"what about refunds"` mid-conversation is unretrievable on its own.

Technique	What it does
Contextualize	Rewrite using conversation history: <i>"What is the enterprise refund policy?"</i>

Technique	What it does
HyDE	Have the LLM write a hypothetical <i>answer</i> , embed that instead — answers resemble documents more than questions do
Multi-query	Generate 3 phrasings, retrieve for each, merge

Citations, and why they matter most

```
context = "\n\n".join(
    f"[{i+1}] (from {meta['source']}, p.{meta['page']})\n{doc}"
    for i, (doc, meta) in enumerate(zip(chunks, metadatas))
)

system = f"""Answer using only the context. Cite sources as [1], [2].

{context}"""
```

💡 Two reasons this is more than a nice touch

- 1. The user can verify.** An unverifiable answer from a system that sometimes hallucinates has limited value.
- 2. You can debug.** When the answer is wrong, citations tell you *instantly* whether retrieval failed or generation failed. Without them you're guessing.

Access control — the enterprise blocker

```
results = collection.query(
    query_embeddings=[q_vec],
    n_results=5,
    where={"department": user.department},    # filter BEFORE the LLM sees it
)
```

🔒 The filter must happen at retrieval, never after

Once a chunk is in the prompt, **the model has seen it**, and no instruction reliably prevents leakage.

And the recurring rule: `user.department` comes from a **verified server session**, not from the request body. Same principle as Topic 39.

Evaluation

"Seems good" isn't a metric. Split by stage, because the fixes are completely different:

Stage	Metric	Question
Retrieval	Recall@K	Is the correct chunk in the top-K?
	Precision	How much of what you retrieved was useful?
Generation	Faithfulness	Is every claim supported by the context?
	Answer relevance	Does it actually answer the question?

🔗 Where pytest earns its place again

Build 30–50 question/answer pairs with known source chunks and run them on every change. RAGAS automates the scoring — but a hand-built set beats no set. Retrieval failures mean chunking, embeddings, or search strategy; generation failures mean prompting or model choice.

Agentic RAG — connecting to Topic 41

Naive RAG always retrieves, **once**. **Agentic RAG makes retrieval a tool** the agent decides to use:

```
tools = [{
  "type": "function",
  "function": {
    "name": "search_docs",
    "description": "Search company documentation for policies, "
                  "procedures, and product information.",
    "parameters": {...},
  },
}]
```

Naive RAG	Agentic RAG
Always retrieves	Skips retrieval for "hello"
Retrieves once	Searches multiple times for complex questions
Accepts bad results	Reformulates and retries
Retrieval only	Combines retrieval with other tools

That's the natural convergence of Topics 41 and 42 — and where most serious systems are heading.

Cost and latency

Each query costs one embedding call (fast, cheap), one vector search (milliseconds), and one LLM call **with 2,000–4,000 extra tokens of context**. That last item dominates.

⚠ More chunks is not better

Tune K down — **4–6 chunks is usually enough**, and more often *hurts*, since models attend poorly to the middle of long contexts. Cache embeddings for repeated queries, and rerank so fewer, better chunks go in.

🎯 Interview Q&A

Q: What is RAG and what problem does it solve?

Retrieval Augmented Generation — retrieving relevant documents at query time and injecting them into the prompt as context. It addresses three limitations of a frozen model: no knowledge of private or internal data, no knowledge past the training cutoff, and a tendency to hallucinate when uncertain. Crucially the model isn't modified; you're changing what's in its context window at inference time.

Q: Walk me through a RAG pipeline.

Two phases. **Indexing** runs offline: load documents, split into overlapping chunks, embed each chunk, store vectors plus metadata in a vector database. **Querying** runs per request: embed the question with the *same* model, retrieve the top-K similar chunks, assemble them into a grounded prompt with instructions to use only that context, and generate. Production systems add hybrid search, reranking, and citations.

Q: How do you choose chunk size?

By balancing context against precision. Too small and chunks lose the surrounding meaning that makes them interpretable; too large and you retrieve mostly irrelevant text that dilutes the signal and burns context window. Around 500–1,000 characters with 10% overlap is a reasonable default, but it depends on document structure — dense legal text differs from conversational FAQs. I'd tune it empirically against a retrieval-recall metric rather than guessing.

Q: RAG or fine-tuning?

RAG for knowledge, fine-tuning for behaviour. RAG suits facts that change, need citation, or require per-user access control — updating means re-indexing a document rather than retraining. Fine-tuning suits consistent style, output format, or domain-specific reasoning patterns that no amount of context will produce. They're complementary: fine-tune for voice, RAG for facts.

Q: Your RAG system gives wrong answers. How do you debug it?

Isolate the stage. Inspect the retrieved chunks first — if the answer isn't in them, it's a **retrieval** problem, and I'd look at chunking strategy, embedding model, and whether the query needs rewriting or hybrid search for exact terms. If the correct chunk was retrieved but the answer is still wrong, it's **generation** — usually a weak grounding instruction, too many distractor chunks, or a model too small to follow the context. Citations make this diagnosis immediate, which is why they're worth building in from the start.

Q: What is hybrid search and why does it matter?

Combining semantic vector search with keyword search like BM25, merging the rankings with something like reciprocal rank fusion. It matters because embeddings capture meaning but handle exact strings poorly — error codes, product SKUs, proper nouns, version numbers. Keyword search nails those and misses paraphrases; together they cover each other's blind spots. It's typically the single biggest retrieval-quality gain over naive vector search.

MEMORY HOOK

RAG = open-book exam for an LLM.

Index once: Load → Chunk → Embed → Store

Query always: Embed → Search → Augment → Generate

Same embedding model both times, or nothing works.